

Aula 07 — Expressões, Atribuição e Controle

Slide 1 — Capa

Aula 07 da disciplina de Paradigmas, abordando expressões, atribuições e estruturas utilizadas para controlar o fluxo dos programas. O conteúdo tem como referência Sebesta, 11ª edição, capítulos 7 e 8.

Slide 2 — Objetivo e relação com situações reais

O objetivo é compreender como as linguagens interpretam aquilo que o programador escreve, garantindo que o código tenha o comportamento esperado. Esse conhecimento ajuda a analisar programas de outras pessoas sem cair em erros de precedência, associatividade ou indentação. Também é importante conhecer os riscos das conversões automáticas, como acontece com `==` em JavaScript e PHP, além de escolher estruturas de controle que deixem o código mais fácil de entender. Alguns casos reais mostram as consequências desses problemas, como o goto fail da Apple em 2014, o erro do Zune relacionado a ano bissexto em 2008 e a comparação de hashes diferentes utilizando `==` no PHP.

Slide 3 — Roteiro da aula

A aula possui aproximadamente 80 minutos e está dividida em seis momentos: quiz inicial, com 10 minutos; expressões, com 20 minutos; conversões e avaliação em curto-circuito, com 15 minutos; atribuição, com 10 minutos; seleção, repetição, goto e comandos protegidos, com 15 minutos; e exercício final para localizar erros, com 10 minutos. A proposta é prática e exploratória: primeiro a turma tenta prever o resultado e depois executa os exemplos no OneCompiler para conferir.

Slide 4 — Quiz de abertura: o que cada código produz?

1. Python — `-4 e 512`: o operador `**` possui prioridade sobre o sinal negativo, portanto `-2 ** 2` significa `-(2 ** 2)`. A exponenciação também é associativa pela direita, então `2 ** 3 ** 2` equivale a `2 ** (3 ** 2)`, resultando em 512.
2. Java — `3345`: primeiro `1 + 2` é calculado como número inteiro, produzindo 3. Depois que aparece a string "3", as operações seguintes passam a ser concatenações, formando "3345".
3. JavaScript — `3 52`: o operador `-` converte "5" para número, produzindo 3. Já o operador `+` encontra uma string e realiza concatenação, formando "52".
4. C — `0: 3 > 2` resulta em 1; depois `1 > 1` é falso, portanto o resultado final é 0.
5. Python — padrão True: como 0 é considerado falso, o `or` retorna o segundo operando. Além disso, `3 > 2 > 1` é uma comparação encadeada equivalente a `(3 > 2) and (2 > 1)`.
6. JavaScript — `true false true`: a igualdade com coerção não é transitiva. Assim, `0 == ""` e `0 == "0"` podem ser verdadeiros, mas `"" == "0"` é falso.

Slide 5 — Expressões aritméticas: questões de projeto

Ao desenvolver uma linguagem, é necessário definir como as expressões serão interpretadas. Entre as principais decisões estão a prioridade dos operadores, a associatividade, a ordem de avaliação dos operandos, o tratamento de efeitos colaterais, a sobrecarga de operadores e a combinação entre diferentes tipos. Um exemplo é $a + b * c$, com $a = 3$, $b = 4$ e $c = 5$. Se tudo fosse calculado da esquerda para a direita, o resultado seria $(3 + 4) * 5 = 35$. Com a precedência tradicional, temos $3 + (4 * 5) = 23$. O APL utiliza avaliação da direita para a esquerda, enquanto o Lisp usa notação prefixa e parênteses para deixar a estrutura da expressão explícita.

Slide 6 — Precedência e associatividade na prática

Python, JavaScript e Java possuem regras diferentes. No Python, $-2 ** 2$ resulta em -4 , enquanto no JavaScript essa escrita gera erro de sintaxe e é necessário usar $(-2) ** 2$. Java não possui o operador $**$. A expressão $2 ** 3 ** 2$ resulta em 512 no Python devido à associação pela direita. Já $1 + 2 + "3"$ gera `TypeError` no Python, mas produz "33" em JavaScript e Java. A divisão também muda entre linguagens: $-7 // 2$ resulta em -4 no Python, JavaScript produz -3.5 com $/$ e Java chega a -3 na divisão inteira. Para o resto, $-7 \% 2$ resulta em 1 no Python e -1 em Java e JavaScript. No Excel, $=-2^2$ resulta em 4. A recomendação é utilizar parênteses sempre que houver possibilidade de dúvida.

Slide 7 — Ordem dos operandos e efeitos colaterais

Considere $a = a + \text{fun1}()$, sabendo que $\text{fun1}()$ altera a para 17 e retorna 3. Se o valor antigo de a for obtido antes da chamada, o cálculo será $5 + 3 = 8$. Se a função for executada primeiro, o resultado poderá ser $17 + 3 = 20$. O Java define a avaliação da esquerda para a direita, produzindo 8. Em C, em determinadas situações, a ordem não é definida pelo padrão. Esse exemplo mostra por que efeitos colaterais tornam as expressões mais difíceis de prever. Quando funções não alteram informações externas, a ordem de avaliação tende a ter menos impacto, o que está relacionado à transparência referencial.

Slide 8 — Operadores sobrecarregados

Um mesmo operador pode ter diferentes significados de acordo com o tipo dos valores. O $+$, por exemplo, pode realizar uma soma ou concatenar strings. O operador $\&$ em C++ pode representar uma operação bit a bit ou ser utilizado para trabalhar com endereços. No Java, $'a' + 1$ resulta em 98, enquanto $"" + 'a' + 1$ produz "a1". O Java não permite que o usuário crie novas sobrecargas de operadores, por isso classes como `BigDecimal` utilizam métodos como `add()` e `multiply()`. No Python, é possível personalizar operadores através de métodos como `__add__()` e `__mul__()`, permitindo que uma classe `Vetor` defina operações como `Vetor(1,2) + Vetor(3,4)`.

Slide 9 — Conversões: alargamento, estreitamento e coerção

O alargamento acontece quando um tipo é convertido para outro com maior capacidade, mas mesmo assim pode ocorrer perda de precisão. Por exemplo, 123456789 pode ser representado de forma aproximada ao ser convertido para float. O estreitamento reduz a capacidade do tipo e geralmente exige conversão explícita. (int) 3.99 produz 3, enquanto (byte) 200 pode resultar em -56. No Java, operações entre byte normalmente resultam em int, por isso `byte s = b + c` não compila diretamente. Também é importante lembrar que `1 / 2` resulta em 0 quando os operandos são inteiros, enquanto `1 / 2.0` resulta em 0.5. C, Java e Python realizam algumas conversões automaticamente, enquanto Go, Rust, ML e F# exigem conversões mais explícitas.

Slide 10 — Coerção implícita: JavaScript e PHP

JavaScript apresenta diversos comportamentos causados pela conversão automática. `"5" - 2` produz 3, `"5" + 2` produz "52", `true + 1` resulta em 2, `[] + {}` pode resultar em "[object Object]" e `"b" + "a" + +"a" + "a"` produz "baNaNaN". No PHP, o chamado *type juggling* também pode causar resultados inesperados. Um exemplo é `"0e123" == "0e456"`, que pode ser verdadeiro porque ambas as strings são interpretadas como números com valor zero. Esse comportamento ficou conhecido por causar problemas na comparação de hashes. Em JavaScript e PHP, uma alternativa mais segura é utilizar `===` e `!==`.

Slide 11 — Expressões relacionais e booleanas

Em C, valores booleanos podem ser representados por inteiros, normalmente 0 para falso e 1 para verdadeiro. Por isso, `3 > 2 > 1` primeiro calcula `3 > 2`, que vale 1, e depois verifica `1 > 1`, resultando em 0. A forma correta seria `(a > b) && (b > c)`. O Python permite comparações encadeadas, então `a > b > c` funciona como esperado. Python, JavaScript e C também possuem vários valores considerados falsos, como 0, vazio ou nulo. Já Java, C#, Go e Rust exigem valores booleanos reais em condições como `if`.

Slide 12 — Avaliação em curto-circuito

Operadores como `&&` e `||` podem interromper a avaliação assim que o resultado já estiver determinado. Isso é útil para evitar erros. No Java, `s != null && s.length() > 0` verifica primeiro se `s` não é nulo e só depois acessa seu tamanho. Com `&`, ambas as partes poderiam ser avaliadas e causar um `NullPointerException`. No JavaScript, `?` permite acessar propriedades de valores que podem ser nulos. O operador `??` só substitui `null` e `undefined`. Assim, se `qtd = 0`, `qtd || 10` resulta em 10, enquanto `qtd ?? 10` mantém 0. Também é preciso ter cuidado com efeitos colaterais, pois em `(a > b) || (b++ / 3)` o `b++` somente será executado quando a primeira condição for falsa.

Slide 13 — Sentenças de atribuição

Em C, a atribuição também pode funcionar como uma expressão. Por isso são possíveis construções como `sum = count = 0` e `while ((ch = getchar()) != EOF)`. O problema é que isso permite erros como `if (x = y)`, em que ocorre uma atribuição no lugar de uma comparação. C, C++ e JavaScript permitem esse comportamento. Java e C# possuem regras mais restritivas, enquanto Python, Go e Rust não permitem a mesma construção. O Python possui atribuição múltipla, como `a, b = b, a`, permitindo trocar os valores sem uma variável auxiliar. Também existe o operador morsa `:=`. Em linguagens funcionais, construções como `val` e `let` geralmente associam nomes a valores de forma mais próxima da imutabilidade.

Slide 14 — Atribuição de modo misto

O Java permite determinadas conversões automáticas quando ocorre alargamento. Assim, `double d = 3` é permitido, mas `int i = 3.7` exige conversão explícita. `(int) 3.7` resulta em 3. Uma situação diferente ocorre em `i += 3.7`, que pode ser aceita devido a uma conversão implícita realizada pelo operador composto. Com `i = 3`, o cálculo chega a 6.7, mas o valor armazenado como inteiro será 6. Em C, as conversões são mais permissivas: `int i = 3.7` pode resultar em 3, `char c = 300` pode resultar em 44 e `unsigned u = -1` pode produzir 4294967295. Outro erro frequente é `acertos / total * 100`, pois a divisão inteira pode acontecer antes da multiplicação. A forma mais segura é `acertos * 100.0 / total`.

Slide 15 — Estruturas de controle: quantas são necessárias?

As estruturas fundamentais de um algoritmo são sequência, seleção e repetição. Böhm e Jacopini demonstraram em 1966 que essas três estruturas são suficientes para representar qualquer algoritmo. Recursos como `for`, `switch` e `do` foram criados para tornar a programação mais prática e facilitar a representação de padrões comuns. Quanto maior a quantidade de recursos diferentes em uma linguagem, maior também pode ser a dificuldade de reconhecer rapidamente códigos escritos por outras pessoas.

Slide 16 — Seleção de dois caminhos: o else pendente

Em C, quando existem vários `if` aninhados, o `else` pertence ao `if` mais próximo que ainda não possui outro `else`. Isso pode gerar uma diferença entre o que a indentação aparenta indicar e o que realmente será executado. Linguagens como Perl, Go e Rust evitam esse problema exigindo o uso de chaves. Algumas linguagens também permitem que uma estrutura condicional produza um valor. No Rust, por exemplo, pode-se usar `let y = if ... else ...`, enquanto no Python existe a forma `x if x > 0 else 2 * x`.

Slide 17 — Seleção múltipla: do switch ao match

O `switch` tradicional do C pode apresentar o problema de queda entre casos. Quando um `break` não é colocado, a execução continua nos próximos casos. Por exemplo,

com `index = 1`, o programa pode executar o primeiro caso, depois o segundo e ainda chegar ao default. Linguagens mais modernas possuem soluções diferentes. Java 14+ oferece `switch` com `->`, evitando a queda automática e permitindo utilizar o `switch` como expressão. Rust possui `match`, que exige que todas as possibilidades sejam consideradas. Go encerra cada caso automaticamente e utiliza `fallthrough` quando a queda é desejada. C# também evita a queda implícita, enquanto Python 3.10 passou a oferecer `match`.

Slide 18 — Laços controlados por contador

Em C, o `for` é uma estrutura de repetição formada por inicialização, condição e atualização. Como a variável pode ser alterada dentro do corpo, é possível modificar a quantidade de repetições. Se o corpo aumentar `i` em 2 e o próprio `for` aumentar mais 1, os valores podem aparecer como 0 3 6 9. No Python, o `for` normalmente percorre um iterável. Por isso, alterar `i` dentro do corpo não modifica a sequência fornecida por `range(10)`, que continuará produzindo 0 até 9. Já `range(10, 0, -3)` gera 10 7 4 1. As decisões de projeto envolvem o tipo da variável, seu escopo, a possibilidade de alteração dentro do corpo e o momento em que os parâmetros são avaliados.

Slide 19 — Laços lógicos, saídas e iteradores

O Java possui `break` rotulado, permitindo abandonar mais de um laço de uma vez. No JavaScript, `for...in` percorre as chaves ou índices, enquanto `for...of` percorre os valores. Em um array com três elementos, `for...in` pode fornecer "0", "1" e "2", fazendo com que uma soma seja transformada em concatenação e produza algo como "0012". Utilizando `for...of`, os valores são realmente somados e o resultado pode ser 60. O caso do Zune também mostra um problema de repetição infinita: em uma determinada situação de ano bissexto com 366 dias, nenhuma condição modificava corretamente os valores e o laço nunca conseguia terminar. O `do-while` deve ser lembrado porque sempre executa seu corpo pelo menos uma vez.

Slide 20 — Desvio incondicional: o goto

O caso `goto fail` da Apple é um exemplo conhecido dos riscos do uso inadequado de desvios incondicionais. Um segundo `goto fail` foi inserido de maneira incorreta e fez com que uma parte da verificação da assinatura fosse ignorada. Dessa forma, a variável de erro permanecia em uma situação que poderia fazer um certificado inválido ser considerado válido. Dijkstra já criticava o uso do `goto` em 1968. C, C++, C# e Go possuem esse comando, enquanto Java não. Recursos como `break`, `continue` e `return` permitem realizar desvios mais controlados. O uso de chaves obrigatórias e ferramentas como `gcc -Wall` também poderia ajudar a encontrar determinados erros.

Slide 21 — Comandos protegidos de Dijkstra

A proposta de Dijkstra utiliza guardas para determinar quando cada alternativa pode ser executada. Todas as condições são avaliadas e, caso várias sejam verdadeiras, qualquer uma delas pode ser escolhida, tornando o resultado não determinístico. No

exemplo de encontrar o maior valor, se dois valores forem iguais, mais de uma alternativa poderá ser válida. Quando nenhuma guarda é verdadeira, pode ocorrer um erro ou o encerramento da repetição, dependendo da estrutura utilizada. Esse conceito influenciou ideias posteriores, como o CSP de Hoare e o select do Go. Quando dois canais estão prontos ao mesmo tempo, o Go pode escolher qualquer um deles.

Slide 22 — Exercício: encontre o bug — 1/3

1. JavaScript: o resultado é Total: 0012. O erro está no uso de for...in, que percorre os índices do array em vez dos valores. Como os índices são strings, ocorre concatenação. A solução é utilizar for...of:

```
for (const p of precos)
    total += p;
```

Assim, o resultado passa a ser Total: 60.

2. Python: o resultado é 180.0, mas o esperado era 200.0. O problema está em d or 10, pois 0 é considerado falso e acaba sendo substituído por 10. A solução é verificar especificamente None:

```
if d is None:
    d = 10
```

Slide 23 — Exercício: encontre o bug — 2/3

3. Java: o resultado é inválido. O erro ocorre porque o switch não possui break, fazendo a execução continuar para os próximos casos. Com dia = 2, o programa passa por "segunda", depois "terça" e chega ao default. Uma alternativa moderna é:

```
String nome = switch (dia) {
    case 1 -> "domingo";
    case 2 -> "segunda";
    case 3 -> "terça";
    default -> "inválido";
};
```

4. C: o programa imprime saldo insuficiente e depois fim, mesmo quando o saldo é suficiente. O motivo é o ; colocado logo após o if, que representa um comando vazio. A correção é retirar o ponto e vírgula e utilizar chaves:

```
if (saque > saldo) {
    printf("saldo insuficiente\n");
}
```

Slide 24 — Exercício: encontre o bug — 3/3

5. Go: a saída é dentro: 20 e depois fora: 10. O problema está no uso de `:=`, que cria uma nova variável dentro do bloco. Para modificar a variável que já existia, deve-se usar `=`:

```
x = x * 2
```

Assim, a saída passa a ser dentro: 20 e fora: 20.

6. Java: a saída é 0.0%, embora o esperado seja 70.0%. Isso acontece porque `acertos / total` é uma divisão inteira. Com `7 / 10`, o resultado é 0, e depois esse valor é convertido para `double`. A solução é realizar a divisão como número real:

```
double taxa = acertos * 100.0 / total;
```

O resultado será 70.0%.

Slide 25 — Síntese: armadilhas e como as linguagens ajudam

Os exemplos mostram diferentes formas de evitar erros. Problemas de precedência podem ser reduzidos com parênteses. Efeitos colaterais podem ser controlados através de funções puras e regras claras de avaliação. Conversões explícitas, como em Go e Rust, diminuem os riscos das conversões automáticas. Em JavaScript e PHP, `===` e `!==` tornam as comparações mais seguras. Exigir valores booleanos no `if` evita condições confusas. Não permitir atribuições como expressões reduz erros como `if (x = y)`. Estruturas modernas de `switch`, `match` e o uso de chaves ajudam a evitar queda entre casos e problemas de `else` pendente. `for...of`, iteradores e mecanismos de saída controlada tornam os laços mais fáceis de compreender. Por fim, `break`, `continue` e `return` são alternativas mais organizadas ao uso indiscriminado de `goto`.

Slide 26 — Atividades complementares

Como pesquisa adicional, podem ser estudadas as formas de expressão utilizadas em Ruby e Lisp, expressões condicionais, implementação de tabelas de desvio em `switch`, utilização de recursão para representar laços em linguagens funcionais, iteradores e blocos em Ruby e o artigo de Dijkstra publicado em 1968. A próxima aula abordará Subprogramas, com base nos capítulos 9 e 10. Uma atividade sugerida é implementar a mesma estrutura de seleção em C, Java, Go e Rust e comparar como cada linguagem trabalha com queda entre casos, exaustividade e facilidade de leitura.